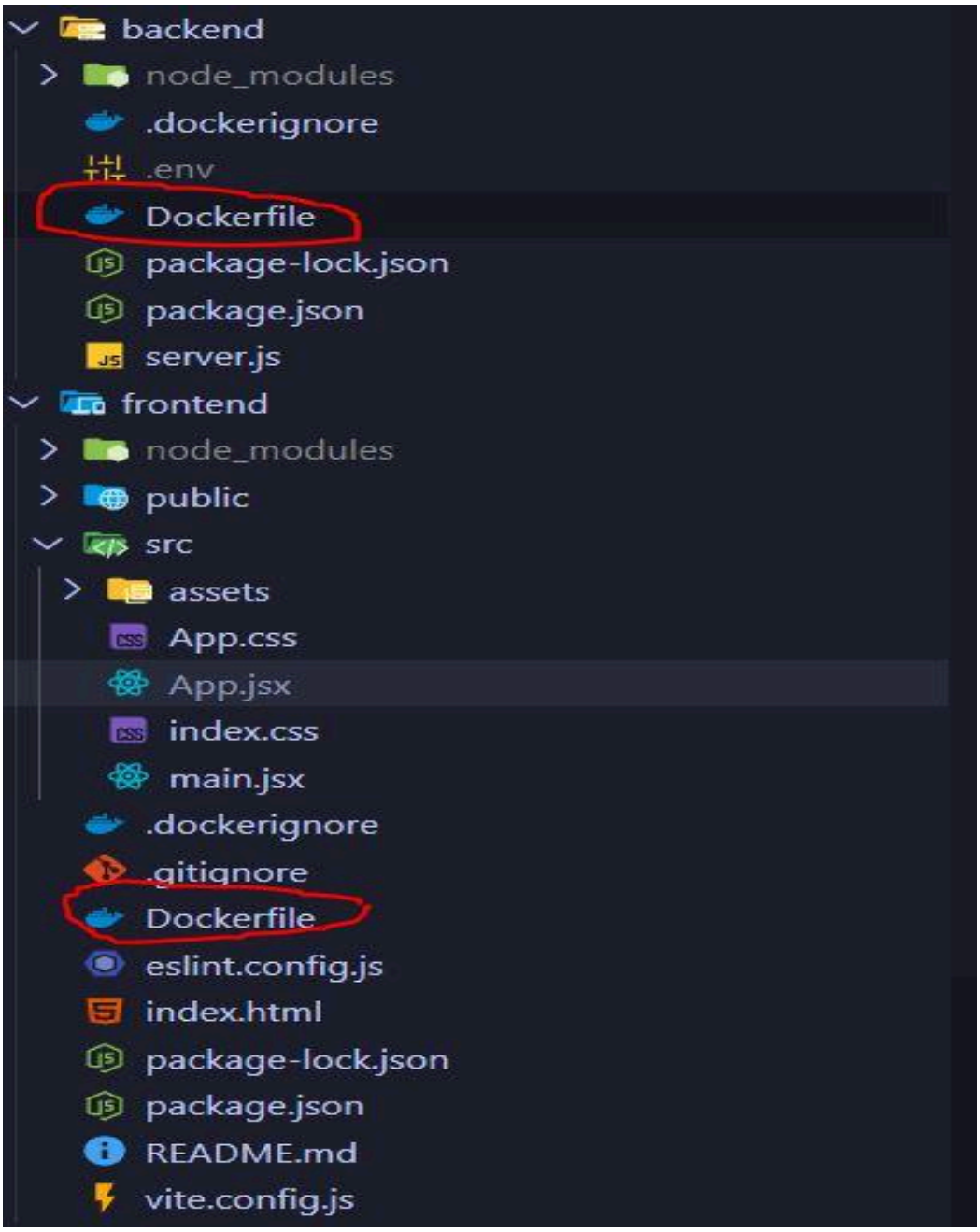
Docker Multi-Container Production Optimized

-Dhanaraj Rajkumar

1. **Folder Setup**: Root project folder contains separate backend and frontend folders

2. **Create *Dockerfile***: Make "***Dockerfile***" (without any extension) on each folder



2.1. Backend Dockerfile:

```
# Build stage
```

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm prune --production #npm prune removes all devDependencies from node_modules and keep only minimal necessary dependencies

# Production stage (Backend)
FROM node:20-alpine
WORKDIR /app
COPY --from=build /app ./
EXPOSE 5000
CMD ["node", "server.js"]
```

2.2. Frontend Dockerfile:

```
#Build Stage
FROM node:20 AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Prodction Stage(Frontend)
#Copy the build(dist) from Stage 1 to the WORKDIR of nginx env
FROM nginx:alpine
WORKDIR /usr/share/nginx/html
COPY --from=build /app/dist ./
EXPOSE 80
#Vite(after built, nginx default port is 80)
#By default, nginx runs CMD ["nginx", "-g", "daemon off;"]
```

3. **Create *.dockerignore***: Make "***.dockerignore***" on each frontend and backend folder.

3.1. `node_modules`

3.2. `.env`

Exclude `node_modules` and `.env` while `COPY . .`

NOTE: Dockerfile is the instruction to docker build on how and what to contain in Docker Image

Frontend Dockerfile ---creates---> Frontend Docker Image

Backend Dockerfile ---creates---> Backend Docker Image

4. **Create** ***docker-compose.yml***: Make "***docker-compose.yml***" file on the root the project.

   Basically it is a **blueprint that tells Docker how to run your entire MERN app as a multi-container system** — instead of manually starting backend and frontend containers one by one.

   Think of it like: *"Start these services, using these images, on these ports, with these environment variables — and make sure they can talk to each other."*

```yaml
version: '3.8'

services:

  backend:
    image: mleonel51/mern-backend:latest #use frontend image pushed in dockerHub
    container_name: mern-backend-container
    restart: always
    ports:
      - "5000:5000"
    env_file:
      - .env #considering i manually ssh into ec2 and insert .env file inside ~/mern-deploy folder so that backend docker container can use it

  frontend:
    image: mleonel51/mern-frontend:latest #use frontend image pushed in dockerHub
    container_name: mern-frontend-container
    restart: always
    ports:
      - "80:80"
    depends_on:
      - backend
```

We directly pull the frontend and backend image from DockerHub using `image:`
This way, we can replace `build:` to prevent building the image inside EC2 Instance because inorder to build latest image inside EC2 we need to copy our latest repo build files ino the EC2 Instance storage which is a lot complex way since its using linux shell command.

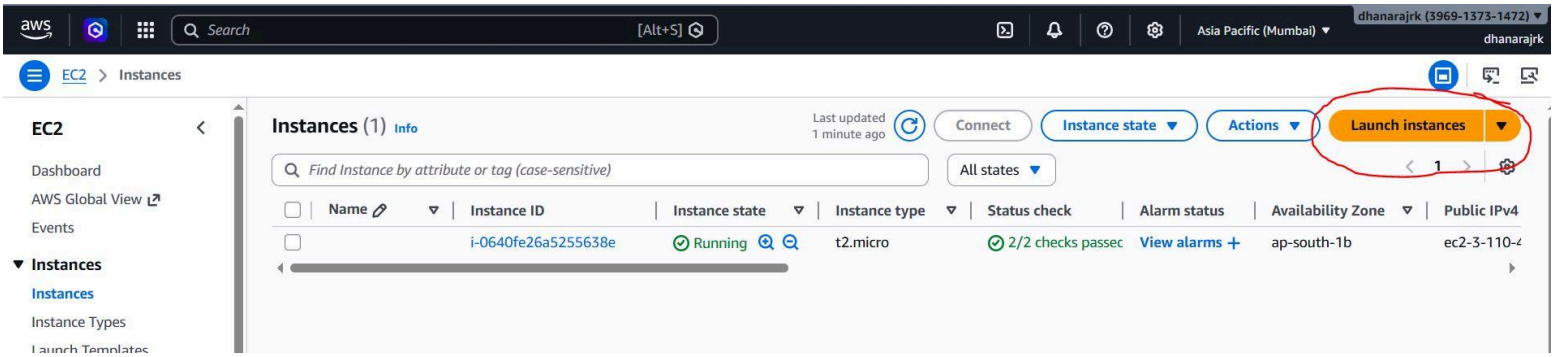
So in later steps we'll build the image and push to DockerHub using github action workflow so that EC2 Instance and just pull it and run docker-compose.

That means we need to put *docker-compose.yml* inside the EC2 project folder ~/mern-deploy/ which we'll later create during Github Action Workflow

Also we need to put .env file inside the EC2 project folder since `backend:` container need it to connect to MONGO_URI and we told *docker-compose.yml* to get it from `env_file:- .env` which is the EC2 project folder ~/mern-deploy/

5. Create EC2 instance and Copy backend .env file to its project folder:

5.1. Create EC2 instance first with below settings:



Launch an instance Info

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags Info

Name

My Web Instance

Add additional tags

Application and OS Images (Amazon Machine Image) Info

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose **Browse more AMIs**.

Search our full catalog including 1000s of application and OS images

Recents | Quick Start

Amazon Linux | macOS | Ubuntu | Windows | Red Hat | SUSE Linux | Debian

Browse more AMIs

Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Amazon Linux 2023 kernel-6.1 AMI
ami-051a31ab2f4d498f5 (64-bit (x86), uefi-preferred) / ami-0d44b036bd2b73294 (64-bit (Arm), uefi)
Virtualization: hvm ENA enabled: true Root device type: ebs

Description

Amazon Linux 2023 (kernel-6.1) is a modern, general purpose Linux-based OS that comes with 5 years of long term support. It is optimized for AWS and designed to provide a secure, stable and high-performance execution environment to develop and run your cloud applications.

Amazon Linux 2023 AMI 2023.10.20260216.1 x86_64 HVM kernel-6.1

Architecture	Boot mode	AMI ID	Publish Date	Username	
64-bit (x86)	uefi-preferred	ami-051a31ab2f4d498f5	2026-02-16	ec2-user	Verified provider

Instance type Info | Get advice

Instance type

t2.micro Free tier eligible
Family: t2 1 vCPU 1 GiB Memory Current generation: true On-Demand Windows base pricing: 0.017 USD per Hour
On-Demand RHEL base pricing: 0.0268 USD per Hour On-Demand Linux base pricing: 0.0124 USD per Hour
On-Demand Ubuntu Pro base pricing: 0.0142 USD per Hour On-Demand SUSE base pricing: 0.0124 USD per Hour

All generations

Compare instance types

Additional costs apply for AMIs with pre-installed software

Key pair (login) Info

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Save

Key pair name - *required*

Select

Create new key pair

Key pair will create* **ec2_instance_key.pem *which is a key to ssh login into our EC2 instance(VM) from our local PC*

Network settings Info

VPC - *required* | Info

vpc-00fb9accea1d05122 (default)
172.31.0.0/16

Subnet | Info

No preference — Create new subnet

Availability Zone | Info

No preference — Enable additional zones

Auto-assign public IP | Info

Enable

Additional charges apply when outside of free tier allowance

Firewall (security groups) | Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

- (●) Create security group
- ( ) Select existing security group

Security group name - *required*

launch-wizard-2

This security group will be added to all network interfaces. The name can't be edited after the security group is created. Max length is 255 characters. Valid characters: a-z, A-Z, 0-9, spaces, and ._-:/()#,@[]+=&;{}!$*

Description - *required* | Info

launch-wizard-2 created 2026-03-01T08:35:14.020Z

Inbound Security Group Rules

Security group rule 1 (TCP, 22, 0.0.0.0/0) — Remove

Type	Protocol	Port range
ssh	TCP	22

Source type	Source	Description - *optional*
Anywhere	Add CIDR, prefix list or security group: 0.0.0.0/0	e.g. SSH for admin desktop

Security group rule 2 (TCP, 443, 0.0.0.0/0) — Remove

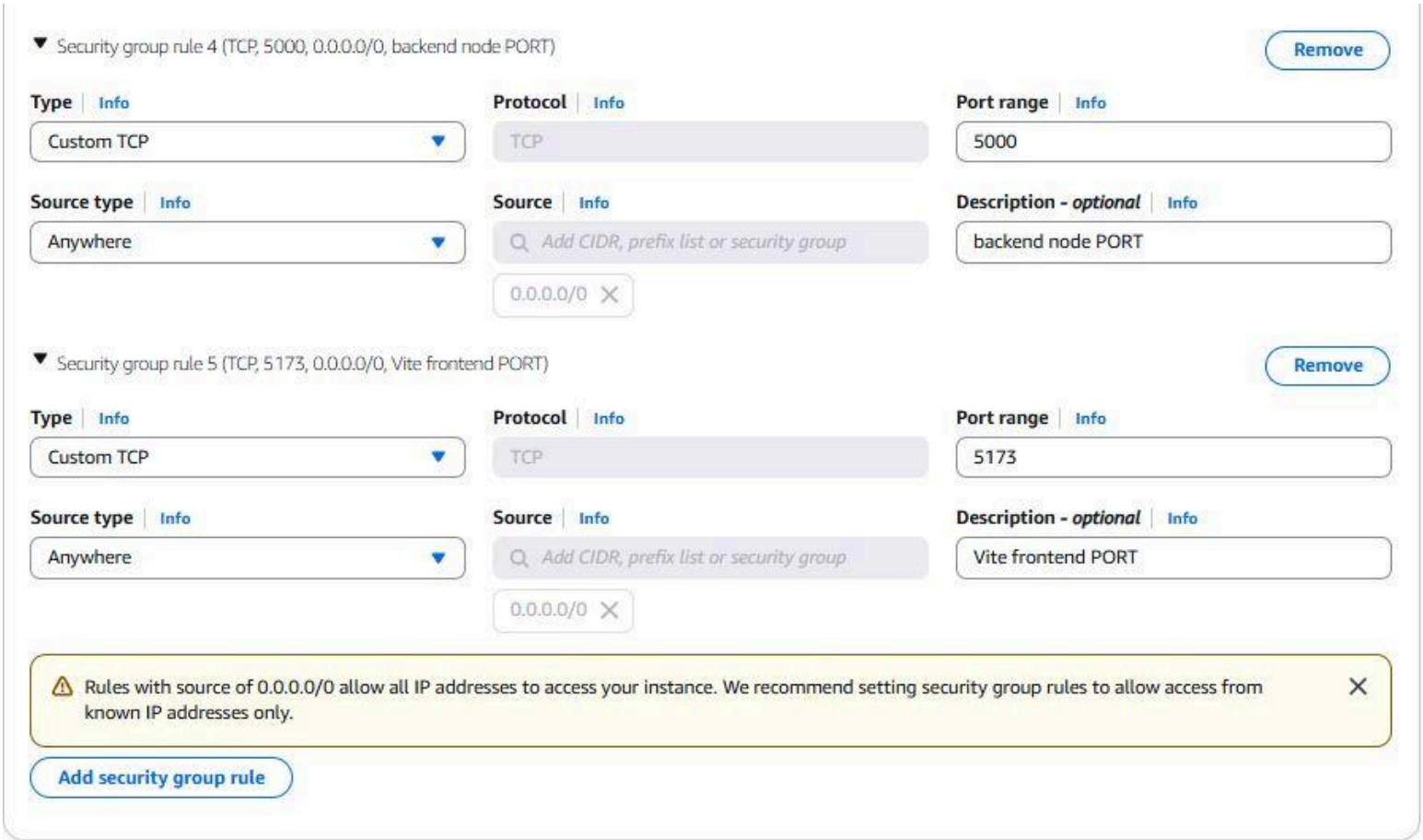
Type	Protocol	Port range
HTTPS	TCP	443

Source type	Source	Description - *optional*
Anywhere	Add CIDR, prefix list or security group: 0.0.0.0/0	e.g. SSH for admin desktop

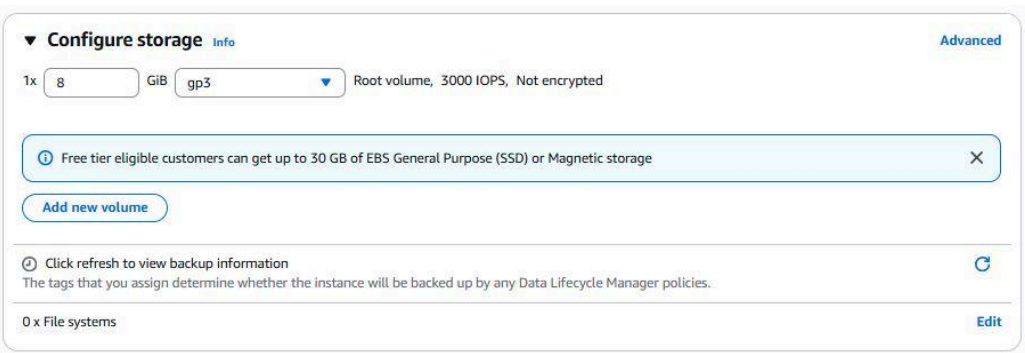
Security group rule 3 (TCP, 80, 0.0.0.0/0) — Remove

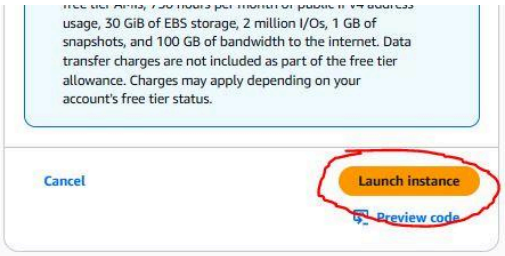
Type	Protocol	Port range
HTTP	TCP	80

Source type	Source	Description - *optional*
Anywhere	Add CIDR, prefix list or security group: 0.0.0.0/0	e.g. SSH for admin desktop



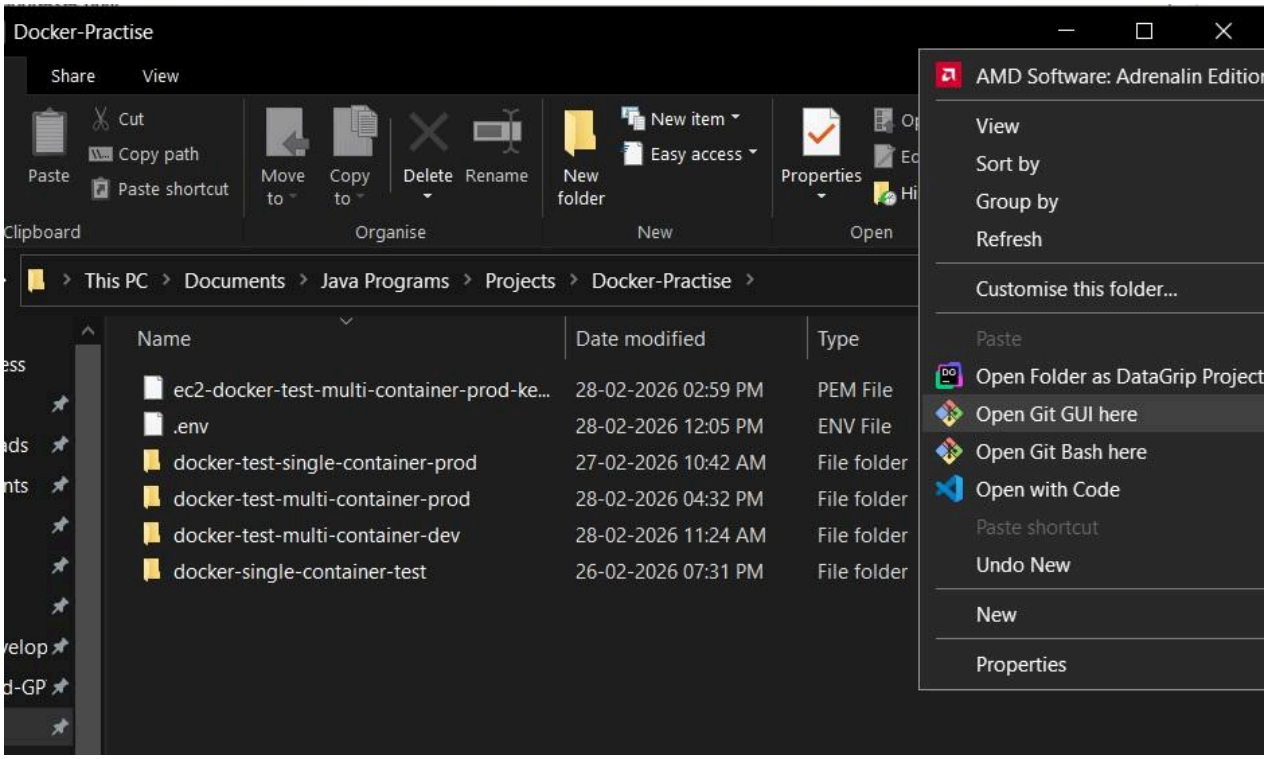
Make TCP PORT range accessible from Anywhere





5.2. SSH(login) into to EC2 instance and insert the backend .env file into the EC2 machine :

- Open Gitbash from the directory where .env is present in local PC



- In Gitbash, type
  `chmod 400 your_ec2_key.pem` Changes file permission: Only file owner can read

  `ssh -i your_ec2_key.pem ec2-user@your_ec2_instance_public_ip`
  Connect to EC2 instance

- After loggin in EC2 virtual machine, type:
  `mkdir ~/mern-deploy` Creates project folder called “mern deploy” inside the EC2 machine

- Open another Gitbash and type:
  `scp -i your-key.pem .env ubuntu@<EC2_IP>:~/mern-deploy/.env` Copies .env from Local PC to EC2 machine *root/mern-deploy/* folder
  You can verify by typing `cat ~/mern-deploy/.env` in EC2

6. **Create *Docker Token Access***: Login Docker on Browser, and go to ***Settings>Personal access token>Generate new token***

   Give any name token name eg. “github-actions”
   Give Read, Write, Delete permission

   Note down the Docker username and Access token key somewhere. We will be using it to login to docker during github action workflow and also inside EC2 script to pull image from DockerHub into EC2 instance

7. **Put Docker and EC2 login credentials to Github Secret Variable**:

8. **Create *github workflow***: in Local Project folder, create *.github/workflows/deploy.yml*

```yaml
name: Build Docker Image, Push Image to Dockerhub & Docker Pull Compose on EC2

on:
  push:
    branches:
      - main

  workflow_dispatch:


jobs:
  build-and-push:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Login to Docker Hub   #use github docker login plugin (which is easier)
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: Build Backend Image  #i dont use github plugin for building and pushing image
cause docker normal command feels easier to me
        run: docker build -t ${{ secrets.DOCKERHUB_USERNAME }}/mern-backend ./backend

      - name: Push Backend Image
```

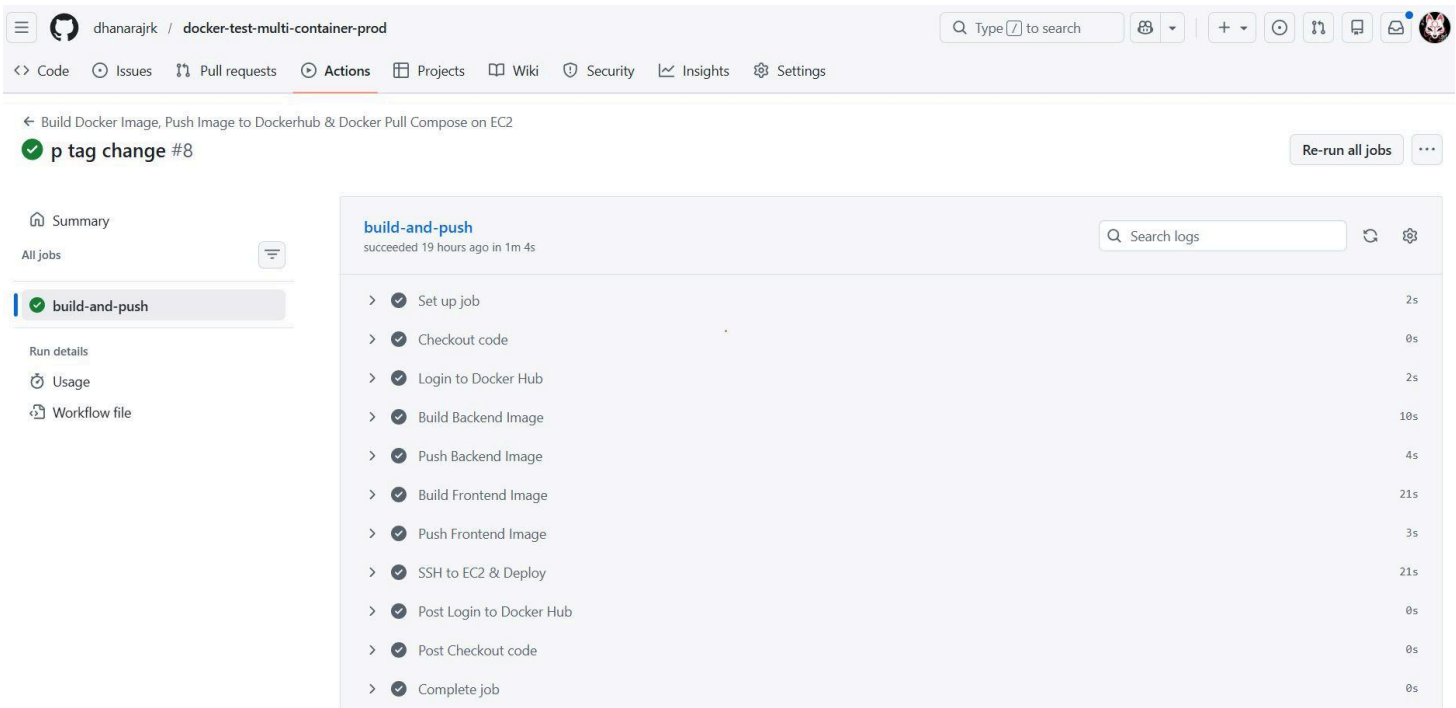
```
        run: docker push ${{ secrets.DOCKERHUB_USERNAME }}/mern-backend

      - name: Build Frontend Image
        run: docker build -t ${{ secrets.DOCKERHUB_USERNAME }}/mern-frontend ./frontend

      - name: Push Frontend Image
        run: docker push ${{ secrets.DOCKERHUB_USERNAME }}/mern-frontend

      - name: SSH to EC2 & Deploy #use appleboy plugin to ssh into AWS EC2 instance and then
pull the image and run compose
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.EC2_HOST }}
          username: ${{ secrets.EC2_USERNAME }}
          key: ${{ secrets.EC2_SSH_KEY }}
          script: |
            # ~ means home directory of ec2, then we cd /mern-deploy OR create one if not
exist before
            cd ~/mern-deploy || mkdir ~/mern-deploy && cd ~/mern-deploy

            # Login to DockerHub on EC2 (needed for private images). We use stdin to hide
password in Github action. Docker will use the stdout password before the echo pipe | symbol
            echo ${{ secrets.DOCKERHUB_TOKEN }} | docker login -u ${{
secrets.DOCKERHUB_USERNAME }} --password-stdin

            # Pull latest docker-compose.yml from github repo
            echo "Pulling latest docker-compose..."
            curl -O
https://raw.githubusercontent.com/dhanarajrk/docker-test-multi-container-prod/main/docker-comp
ose.yml

            # Pull latest images from DockerHub
            docker-compose pull

            # Destroy old containers
            docker-compose down

            # Start new containers (this is not building cause we already built and uploaded
image to Dockerhub. This is just downloading image and running as Docker Container inside EC2
VM)
            docker-compose up -d

            # Removes old unused images from EC2 after running the new images TO SAVE SPACE
            docker image prune -a -f
```

9. **<u>Update API endpoints to EC2 instance IP address</u>**: Replace “localhost” to “EC2_Public_IPv4” because EC2 is the new hosting PC now.

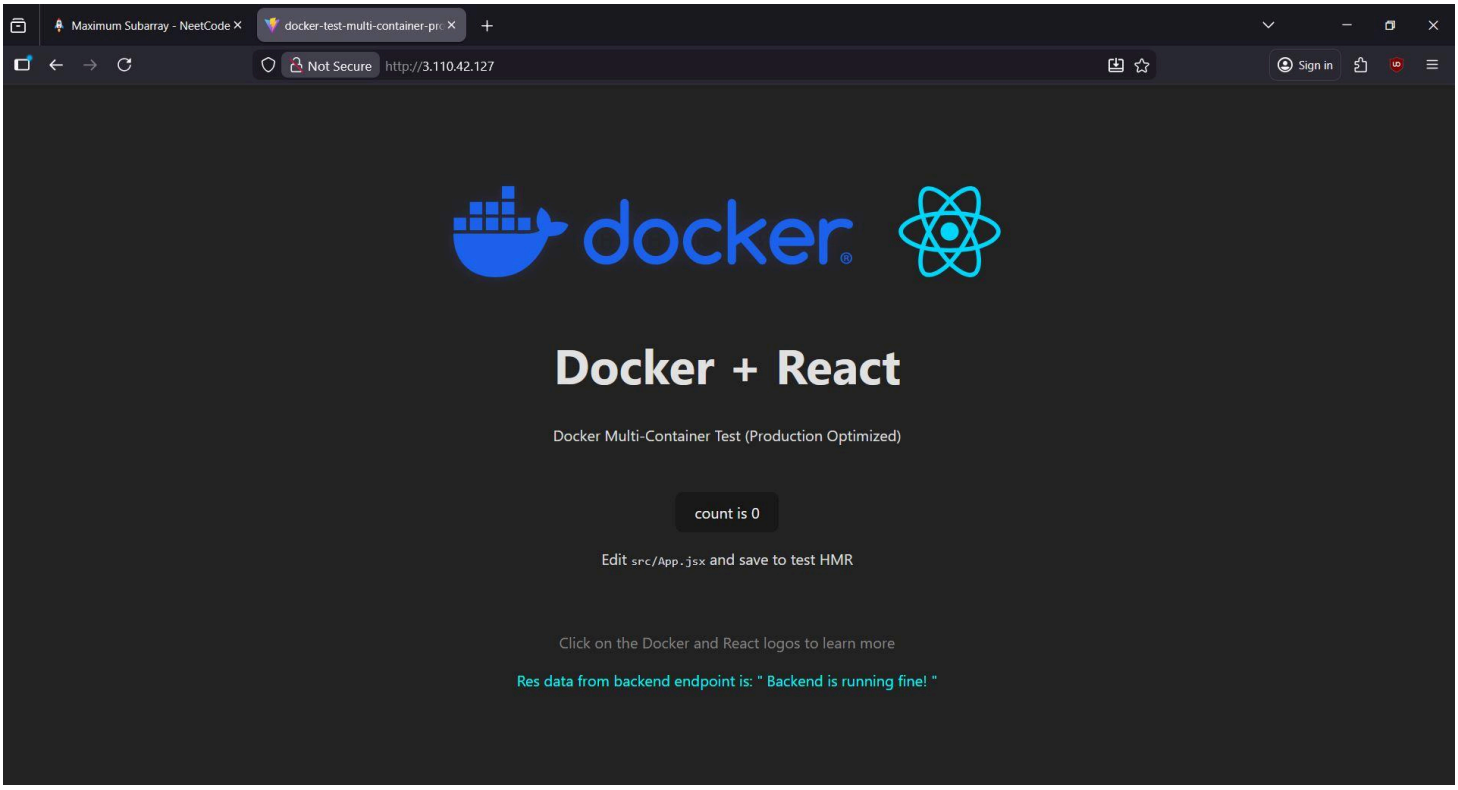
Example:

```
useEffect(()=>{
    //fetch("http://localhost:5000/") //works for Local Dev
    fetch("http://3.110.42.127:5000/") //Replaced localhost with EC2 instance Public ipv4
address cause that VM is a PC now and its name is identified as 3.110.42.127 not as localhost.
    .then(res => res.text())
    .then(data => setBackendMsg(data));
  }, []);
```

10. **<u>Run the github workflow</u>**: push our updated project to github repo, it will automatically trigger the workflow and deploy our app to EC2 instance.



11. **<u>Visit EC2 instance Public Address on the browser</u>**: The App is Live!!


Maximum Subarray - NeetCode
docker-test-multi-container-pr
Not Secure
http://3.110.42.127
Sign in
docker
Docker + React
Docker Multi-Container Test (Production Optimized)
count is 0
Edit src/App.jsx and save to test HMR
Click on the Docker and React logos to learn more
Res data from backend endpoint is: " Backend is running fine! "